

1 Co-word and Keyword Co-occurrence

1.1 Learning objectives

After completing this chapter, you will be able to:

- Explain the principles of co-word analysis and its role in science mapping
- Extract and clean keywords from OpenAlex data (author keywords and concepts)
- Build a keyword co-occurrence matrix and convert it to a network
- Apply community detection to identify topical clusters
- Visualise keyword co-occurrence networks with interpretable layouts

1.2 Setup

```
library(tidyverse)
library(openalexR)
library(igraph)
library(tidygraph)
library(ggraph)
library(glue)
library(gt)

set.seed(20260509)

source(here::here("R", "api_helpers.R"))
source(here::here("R", "utils.R"))
source(here::here("R", "sci_palette.R"))
```

1.3 Conceptual background

Co-word analysis maps the conceptual structure of a research field by examining which terms appear together in the same documents. Introduced by @callon1983, the method assumes that when two keywords repeatedly co-occur across publications, they are thematically related. The resulting co-occurrence network reveals the topical landscape of a field: clusters of densely connected keywords represent coherent research themes, while bridges between clusters indicate interdisciplinary connections.

The method works with several types of terms:

- **Author keywords:** Terms selected by the paper’s authors. These are intentional descriptors but suffer from inconsistency — authors may use different terms for the same concept (“machine learning” vs. “ML” vs. “statistical learning”).
- **Indexed keywords:** Terms assigned by database indexers (e.g., MeSH terms in PubMed). More consistent but only available in some databases.
- **OpenAlex concepts:** Algorithmically assigned topics at multiple levels of a hierarchical taxonomy. These provide broad coverage but may miss nuanced distinctions [@priem2022].
- **Title/abstract words:** Extracted via text mining. Comprehensive but noisy; requires extensive preprocessing.

Normalisation is important for co-word networks. Raw co-occurrence counts favour high-frequency terms. Common normalisations include the **association strength** (equivalent to pointwise mutual information) and

the **equivalence index** (cosine of the co-occurrence vector). @waltman2012 demonstrated that association strength produces the most balanced network structures for bibliometric mapping.

Co-word analysis complements citation-based methods (??). Citation networks reveal intellectual influence; co-word networks reveal thematic content. Combining both provides a more complete picture of a field's structure.

1.4 Worked example

1.4.1 Extracting keywords

We extract author keywords from a sample of scientometrics research.

```
works <- oa_fetch(
  entity = "works",
  primary_location.source.id = "S148561398",
  from_publication_date = "2019-01-01",
  to_publication_date = "2023-12-31",
  options = list(sample = 500, seed = 42)
)
```

```
keywords <- works |>
  select(id, topics) |>
  unnest(topics, names_sep = "_") |>
  filter(topics_display_name != "") |>
  select(work_id = id, keyword = topics_display_name) |>
  mutate(keyword = str_to_lower(str_trim(keyword))) |>
  filter(!is.na(keyword), nchar(keyword) >= 3)
```

```
kw_counts <- keywords |>
  count(keyword, sort = TRUE)
```

```
cat(glue("Total keyword occurrences: {nrow(keywords)}\n"))
```

```
#> Total keyword occurrences: 5196
```

```
cat(glue("Unique keywords: {n_distinct(keywords$keyword)}\n"))
```

```
#> Unique keywords: 412
```

```
kw_counts |>
  head(20) |>
  mutate(keyword = fct_reorder(keyword, n)) |>
  ggplot(aes(x = n, y = keyword)) +
  geom_col(fill = palette_sci(1)) +
  labs(x = "Frequency", y = NULL) +
  theme_sci()
```

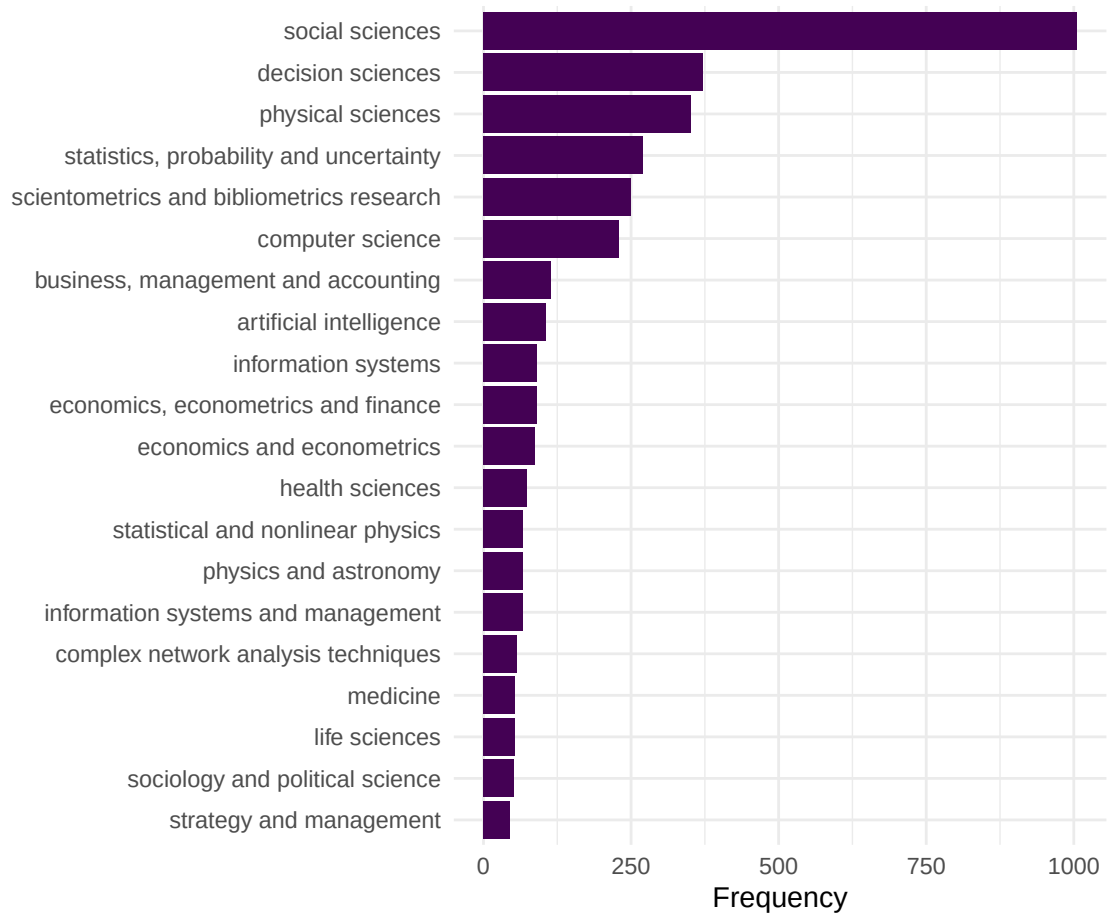


Figure 1: Top 20 most frequent keywords in the Scientometrics sample.

1.4.2 Building the co-occurrence network

We create edges between keywords that appear in the same paper.

```
kw_frequent <- kw_counts |>
  filter(n >= 5) |>
  pull(keyword)

kw_filtered <- keywords |>
  filter(keyword %in% kw_frequent)

coword_pairs <- kw_filtered |>
  inner_join(kw_filtered, by = "work_id", suffix = c("_a", "_b"),
    relationship = "many-to-many") |>
  filter(keyword_a < keyword_b) |>
  count(keyword_a, keyword_b, name = "cooccurrence")

coword_top <- coword_pairs |>
  filter(cooccurrence >= 3)

g_kw <- graph_from_data_frame(
  coword_top |> select(keyword_a, keyword_b, weight = cooccurrence),
  directed = FALSE
) |>
  simplify(edge.attr.comb = list(weight = "sum"))

cat(glue("Keyword network: {vcount(g_kw)} nodes, {ecount(g_kw)} edges\n"))
```

```
#> Keyword network: 114 nodes, 913 edges
```

1.4.3 Community detection for topical clusters

```
V(g_kw)$degree <- degree(g_kw)
V(g_kw)$strength <- strength(g_kw)

communities <- cluster_leiden(g_kw, resolution_parameter = 0.8,
  objective_function = "modularity")
V(g_kw)$community <- as.factor(membership(communities))

cat(glue("Communities: {length(unique(membership(communities)))}\n"))
```

```
#> Communities: 3
```

```
cat(glue("Modularity: {round(modularity(g_kw, membership(communities))), 3}\n"))
```

```
#> Modularity: 0.236
```

```
community_summary <- tibble(
  keyword = V(g_kw)$name,
  community = V(g_kw)$community,
```

community	top_keywords
1	social sciences, decision sciences, statistics, probability and uncertainty, scientometrics and h
2	physical sciences, computer science, artificial intelligence, information systems, physics and a
3	health sciences, medicine, public health, environmental and occupational health, health and

```

strength = V(g_kw)$strength
) |>
group_by(community) |>
slice_max(strength, n = 5) |>
summarise(top_keywords = paste(keyword, collapse = ", "),
          n_keywords = n(), .groups = "drop")

community_summary |> gt()

```

1.4.4 Visualisation

```

set.seed(42)
layout <- create_layout(as_tbl_graph(g_kw), layout = "fr")

ggraph(layout) +
  geom_edge_link(aes(width = weight), alpha = 0.15, colour = "grey60") +
  scale_edge_width_continuous(range = c(0.3, 2), guide = "none") +
  geom_node_point(aes(size = degree, colour = community), alpha = 0.8) +
  geom_node_text(aes(label = ifelse(degree > quantile(degree, 0.85),
                                         name, NA_character_)),
                repel = TRUE, size = 2.5, max.overlaps = 20, na.rm = TRUE) +
  scale_size_continuous(range = c(1, 6), guide = "none") +
  scale_colour_manual(values = palette_sci(
    n_distinct(V(g_kw)$community)
  )) +
  labs(colour = "Cluster") +
  theme_void(base_family = "sans", base_size = 11) +
  theme(legend.position = "bottom")

```

1.5 Diagnostics and interpretation

- **Keyword cleaning:** Inconsistent terminology inflates the vocabulary and creates spurious nodes. Standardise spelling, merge synonyms (e.g., “h-index” and “hirsch index”), and remove generic terms (“research”, “analysis”) that co-occur with everything but convey no topical information.
- **Frequency threshold:** Including rare keywords produces large, sparse, unreadable networks. Start with keywords appearing in at least 5 papers and adjust based on corpus size.
- **Community interpretability:** Each community should correspond to a recognisable research theme. If communities are uninterpretable, the resolution parameter may need adjustment or keywords need further cleaning.
- **Centrality interpretation:** High-degree keywords are thematically central (used across many contexts). High-betweenness keywords bridge distinct topics and may represent interdisciplinary concepts.

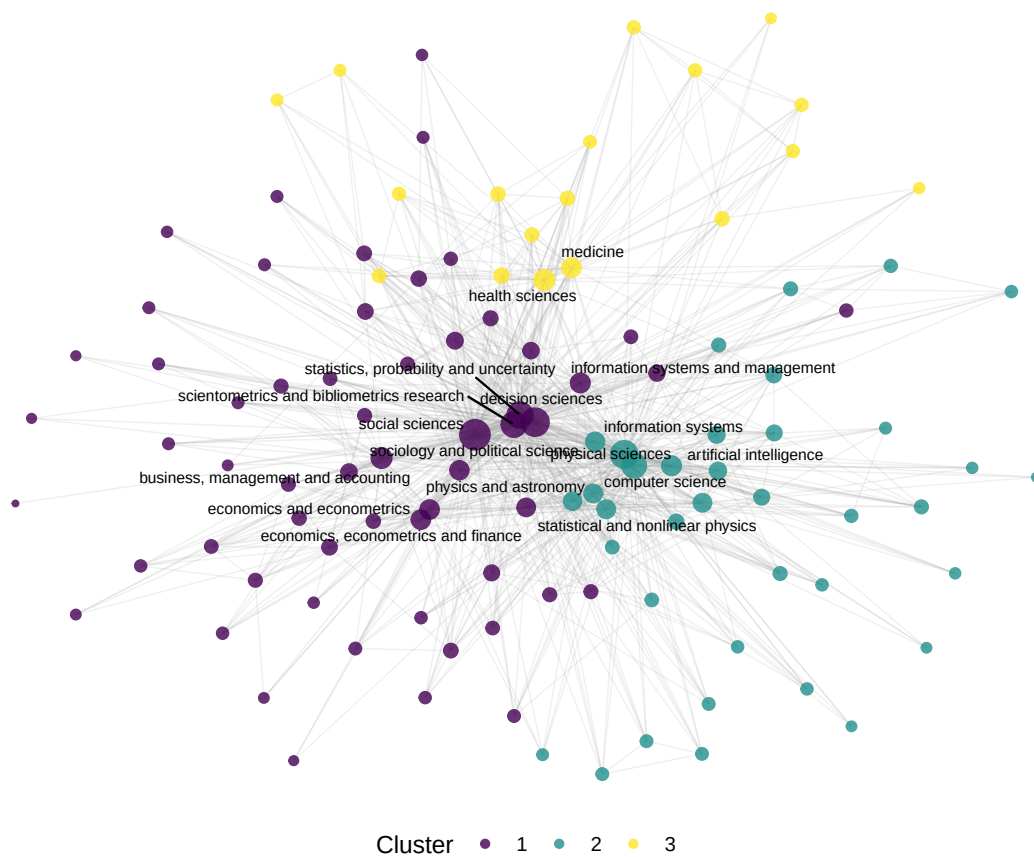


Figure 2: Keyword co-occurrence network coloured by topical cluster.

1.6 Limitations and responsible use

1.7 Limitations and responsible use

- **Vocabulary inconsistency.** Author keywords are not controlled vocabulary. The same concept may appear under multiple terms, fragmenting what should be a single node. Merging synonyms requires domain expertise.
- **Algorithmic concepts.** OpenAlex concepts are assigned by machine learning and may contain errors — especially at fine-grained levels. Always validate topic assignments by reading sample papers [priem2022].
- **Static snapshots.** A co-word network represents a time-averaged view. Emerging topics with few publications may be invisible. Consider building temporal slices to track field evolution.
- **Not evaluative.** Popular keywords indicate research activity, not research quality or societal impact. Do not equate topical centrality with importance [hicks2015].

1.8 Common pitfalls

1.9 Common pitfalls

- **Not cleaning keywords.** Punctuation variants (“co-authorship” vs. “coauthorship”), case differences, and trailing whitespace create duplicate nodes. Clean before building the network.
- **Including stop-keywords.** Generic terms like “research”, “study”, “analysis” co-occur with everything and dominate the network without conveying topical information. Remove them.
- **Comparing co-occurrence counts across corpora of different sizes.** Larger corpora produce higher co-occurrence counts mechanically. Normalise or use relative measures.
- **Over-reading small clusters.** A cluster with two or three keywords may reflect a single paper, not a research theme. Report cluster size alongside content.

1.10 Exercises

1. **Author keywords vs. concepts.** Build co-occurrence networks from both author keywords (if available) and OpenAlex concepts for the same corpus. Compare the resulting topical clusters.
2. **Temporal evolution.** Split the corpus into yearly slices. For each year, build a co-word network and identify the top 5 keywords by degree. Which keywords are consistently central, and which emerge over time?
3. **Strategic diagram.** Compute density (internal cohesion) and centrality (external connections) for each keyword cluster. Plot them on a strategic diagram (density vs. centrality quadrants). Which clusters are core themes and which are peripheral?
4. **Normalisation comparison.** Build two networks from the same data: one with raw co-occurrence counts and one with association strength normalisation. How do the most central keywords differ?

1.11 Solutions

Solutions are provided in 1.11.

1.12 Further reading

- @callon1983 — The foundational paper on co-word analysis for mapping science.
- @waltman2012 — Unified framework for bibliometric network construction, including keyword networks.
- @aria2017 — `bibliometrix` includes co-word analysis via the `biblioNetwork()` function.
- @priem2022 — OpenAlex concepts and topic classification.

1.13 Session info

```
sessionInfo()
```

```
#> R version 4.4.1 (2024-06-14)
#> Platform: x86_64-pc-linux-gnu
#> Running under: Ubuntu 24.04.4 LTS
#>
#> Matrix products: default
#> BLAS: /usr/lib/x86_64-linux-gnu/openblas-pthread/libblas.so.3
#> LAPACK: /usr/lib/x86_64-linux-gnu/openblas-pthread/libopenblas-p-r0.3.26.so; LAPACK version 3.12.0
#>
#> locale:
#>  [1] LC_CTYPE=C.UTF-8      LC_NUMERIC=C          LC_TIME=C.UTF-8
#>  [4] LC_COLLATE=C.UTF-8    LC_MONETARY=C.UTF-8   LC_MESSAGES=C.UTF-8
#>  [7] LC_PAPER=C.UTF-8      LC_NAME=C             LC_ADDRESS=C
#> [10] LC_TELEPHONE=C        LC_MEASUREMENT=C.UTF-8 LC_IDENTIFICATION=C
#>
#> time zone: UTC
#> tzcode source: system (glibc)
#>
#> attached base packages:
#> [1] stats      graphics  grDevices  utils      datasets  methods    base
#>
#> other attached packages:
#>  [1] ggraph_2.2.2      tidygraph_1.3.1    igraph_2.3.2      quanteda_4.4
#>  [5] pdftools_3.9.0    arrow_24.0.0       bibliometrix_5.4.0 RefManageR_1.4.0
#>  [9] bib2df_1.1.2.0    rcrossref_1.2.1    gt_1.3.0          tidytext_0.4.3
#> [13] glue_1.8.1        openalexR_3.0.1    lubridate_1.9.5    forcats_1.0.1
#> [17] stringr_1.6.0     dplyr_1.2.1        purrr_1.2.2        readr_2.2.0
#> [21] tidyr_1.3.2       tibble_3.3.1       ggplot2_4.0.3      tidyverse_2.0.0
#> [25] bookdown_0.46     fs_2.1.0           rmarkdown_2.31
#>
#> loaded via a namespace (and not attached):
#>  [1] bibtex_0.5.2      RColorBrewer_1.1-3 jsonlite_2.0.0
#>  [4] magrittr_2.0.5    farver_2.1.2       vctrs_0.7.3
#>  [7] memoise_2.0.1     askpass_1.2.1      base64enc_0.1-6
#> [10] tinytex_0.59      htmltools_0.5.9    contentanalysis_1.0.0
#> [13] curl_7.1.0        janeaustenr_1.0.0  cellranger_1.1.0
#> [16] htmlwidgets_1.6.4 tokenizers_0.3.0    plyr_1.8.9
#> [19] httr2_1.2.2       cachem_1.1.0       plotly_4.12.0
#> [22] dimensionsR_0.0.3 mime_0.13           lifecycle_1.0.5
#> [25] pkgconfig_2.0.3   Matrix_1.7-0       R6_2.6.1
#> [28] fastmap_1.2.0     shiny_1.13.0       digest_0.6.39
```

```

#> [31] shinycssloaders_1.1.0  rprojroot_2.1.1      SnowballC_0.7.1
#> [34] labeling_0.4.3         urltools_1.7.3.1     timechange_0.4.0
#> [37] polyclip_1.10-7        httr_1.4.8           compiler_4.4.1
#> [40] here_1.0.2             bit64_4.8.0          withr_3.0.2
#> [43] S7_0.2.2              backports_1.5.1      viridis_0.6.5
#> [46] ggforce_0.5.0          MASS_7.3-60.2        rappdirs_0.3.4
#> [49] bibliometrixData_0.3.0 tools_4.4.1          otel_0.2.0
#> [52] stopwords_2.3          zip_2.3.3            httpuv_1.6.17
#> [55] rentrez_1.2.4          promises_1.5.0        grid_4.4.1
#> [58] stringdist_0.9.17     generics_0.1.4        gtable_0.3.6
#> [61] tzdb_0.5.0            rscopus_0.9.0        ca_0.71.1
#> [64] data.table_1.18.4     hms_1.1.4            xml2_1.5.2
#> [67] utf8_1.2.6            ggrepel_0.9.8        pillar_1.11.1
#> [70] later_1.4.8           tweenr_2.0.3          lattice_0.22-6
#> [73] bit_4.6.0             tidyselect_1.2.1     miniUI_0.1.2
#> [76] knitr_1.51            gridExtra_2.3         crul_1.6.0
#> [79] xfun_0.57             graphlayouts_1.2.3   DT_0.34.0
#> [82] humaniformat_0.6.0    visNetwork_2.1.4     stringi_1.8.7
#> [85] lazyeval_0.2.3        qpdf_1.4.1           yaml_2.3.12
#> [88] evaluate_1.0.5        codetools_0.2-20     httpcode_0.3.0
#> [91] cli_3.6.6            xtable_1.8-8         dichromat_2.0-0.1
#> [94] Rcpp_1.1.1-1.1        readxl_1.4.5         triebeard_0.4.1
#> [97] XML_3.99-0.23         parallel_4.4.1       assertthat_0.2.1
#> [100] pubmedR_1.0.2         viridisLite_0.4.3    scales_1.4.0
#> [103] openxlsx_4.2.8.1      rlang_1.2.0          fastmatch_1.1-8

```